ECE 560: EMB. SYS. ARCHITECTURES

FINAL PROJECT: SHIELDS UP! REPORT TEMPLATE V1.0

Archit Kulkarni, Ishan Deshmukh

aakulka2@ncsu.edu , isdeshmu@ncsu.edu

NOTE: Your responses for questions 2 and 3 must be entered in the CSV spreadsheet.

INTRODUCTION

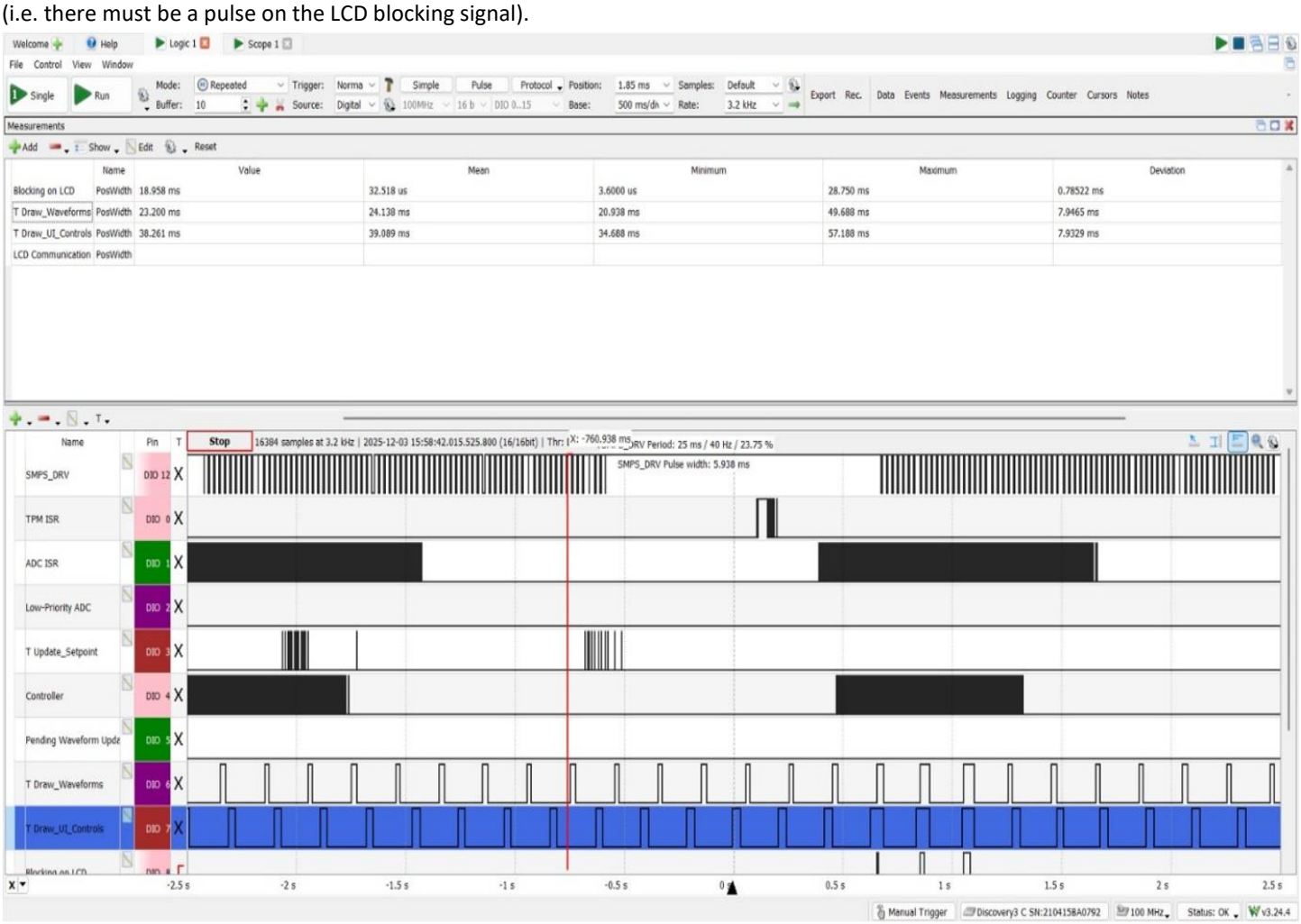
This report documents the implementation and analysis of an RTOS-based embedded control system for LED current regulation. The project involves synchronization mechanisms for mutual exclusion, oscilloscope-style waveform display, and fault detection and management strategies.

SYNCHRONIZATION

Mutual Exclusion for LCD Update Operations

TIMING ANALYSIS FOR BASE CODE

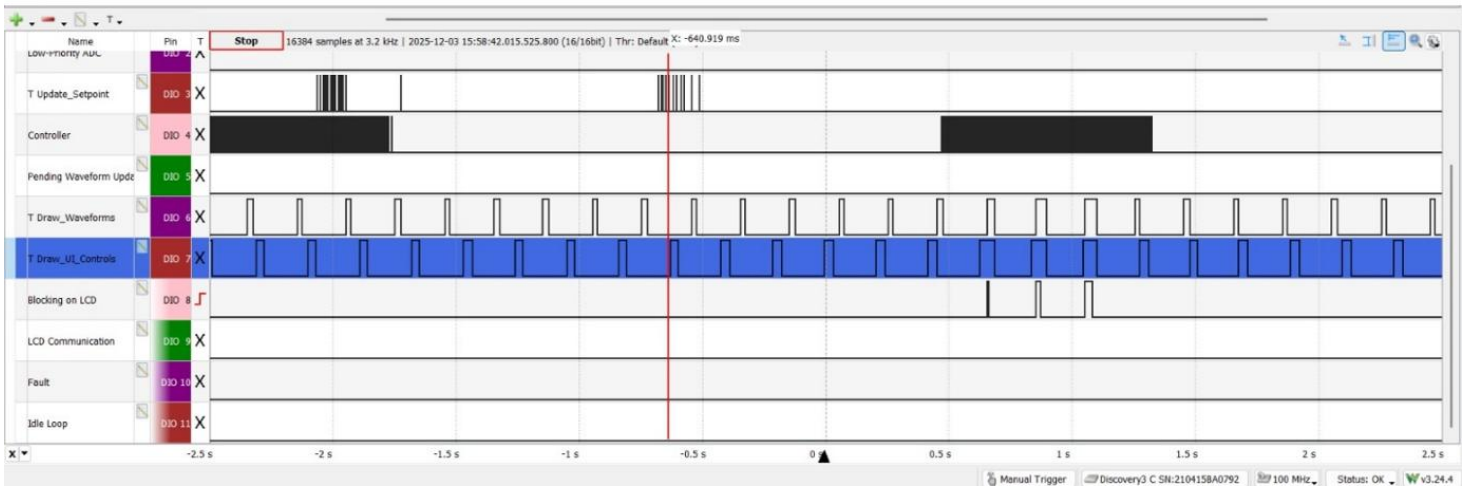
1. Provide two screenshots of the **logic analyzer** with trace and measurement windows showing each thread accessing the mutex (i.e. there must be a pulse on the LCD blocking signal).



Measurements

Add Show Edit Reset

Name		Value	Mean	Minimum	Maximum	Deviation
Blocking on LCD	PosWidth	18.958 ms	32.518 us	3.6060 us	28.750 ms	0.78522 ms
T Draw_Waveforms	PosWidth	23.200 ms	24.138 ms	20.938 ms	49.688 ms	7.9465 ms
T Draw_UI_Controls	PosWidth	38.261 ms	39.089 ms	34.688 ms	57.188 ms	7.9329 ms
LCD Communication	PosWidth					



2. **Timing Statistics Table.**

Measurement	Minimum	Average	Maximum
Thread_Draw_Waveforms: Time blocking on LCD mutex	10.458 ms	12.134 ms	28.750 ms
Thread_Draw_Waveforms: Thread execution time	21.325 ms	24.139 ms	49.088 ms
Thread_Draw_UI_Controls: Time blocking on LCD mutex	10.980 ms	24.309 ms	57.186 ms
Thread_Draw_UI_Controls: Thread execution time	5.783 ms	6.902 ms	7.932 ms

3. **Timing Statistics Analysis:**

The timing statistics reveal several key relationships:

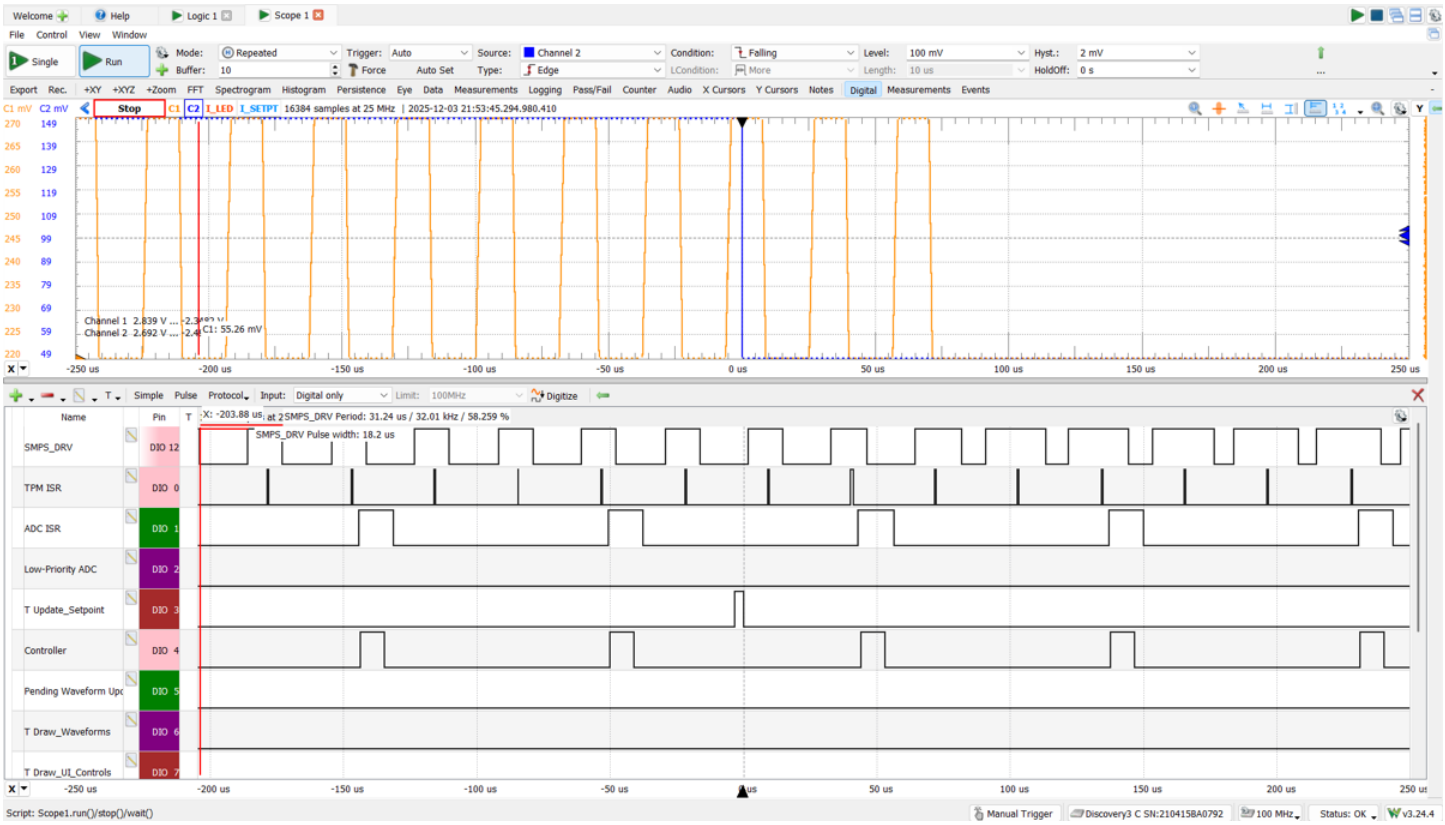
- The minimum blocking time (10.458 ms for Thread_Draw_Waveforms and 10.980 ms for Thread_Draw_UI_Controls) represents the mutex access overhead when the LCD is available. This overhead is consistent across both threads, indicating predictable RTOS mutex behavior.
- The Thread_Draw_Waveforms has significantly longer average execution time (24.139 ms) compared to Thread_Draw_UI_Controls (6.902 ms), which is expected since drawing waveforms involves rendering 960 samples across two channels, while the UI controls update is simpler.
- The maximum blocking times (28.750 ms and 57.186 ms) occur when one thread must wait for the other to complete its LCD operations. The higher maximum for Thread_Draw_UI_Controls suggests it occasionally waits for the waveform drawing to complete.
- No unexpected statistics were observed. The relationship between blocking time and execution time follows the expected pattern: blocking time + actual work time ≈ total execution time when contention occurs.

Synchronization for Scope Waveform Display

With RTOS (Event Flags Approach)

I implemented the RTOS approach using event flags to synchronize the ADC ISR and Thread_Draw_Waveforms for oscilloscope-style waveform display. When the setpoint current crosses the trigger threshold, the ISR begins filling the sample buffers with measured and setpoint current values. Once the buffers are completely filled (960 samples), the ISR calls osEventFlagsSet() to signal the SCOPE_BUFFERS_FULL_FLAG event. The thread periodically checks this flag using osEventFlagsWait() with a non-blocking timeout, and when the flag is set, it acquires the LCD mutex, draws the waveforms, and clears the pending waveform debug signal.

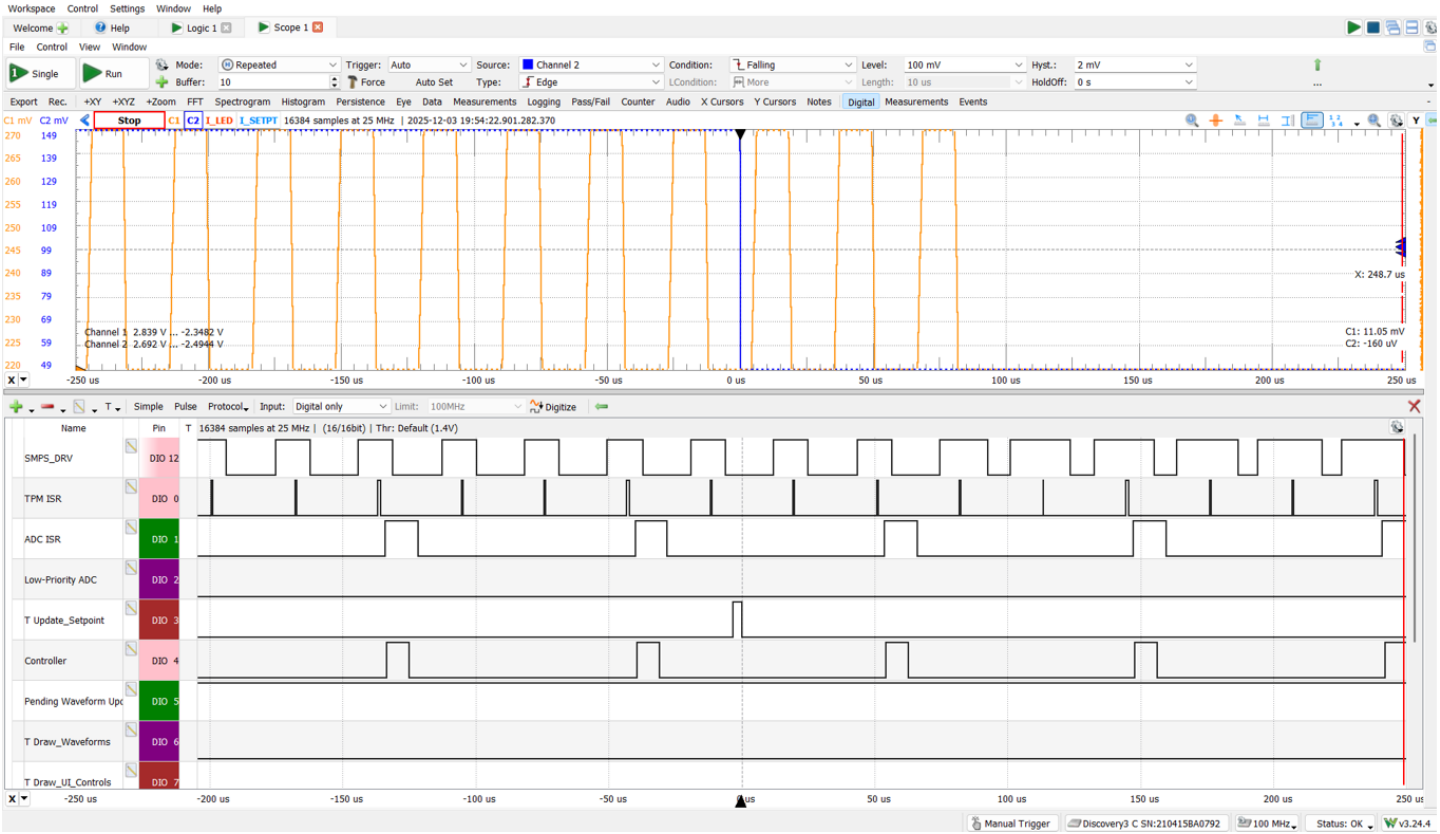
This approach provides clean separation between the ISR's data acquisition and the thread's display rendering, leveraging the RTOS's built-in synchronization to avoid race conditions while maintaining stable, triggered waveform displays. The logic analyzer output confirms proper operation, showing the Pending Waveform Update signal (DIO 5) going high during buffer filling and clearing after the thread completes drawing, demonstrating successful synchronization between the high-priority ISR and lower-priority display thread.



Without RTOS (State Machine Approach)

We implemented the state machine approach for scope synchronization between the ADC ISR and the drawing thread. This method uses the existing SCOPE_STATE_E enum with four states: Armed (waiting for trigger), Triggered (filling buffers), Full (ready to draw), and Disabled (thread drawing). The ADC ISR waits until the LED current setpoint crosses a 50 mA threshold, then fills the sample buffers with 960 samples before transitioning to the Full state. The drawing thread checks for this state, transitions to Disabled to prevent buffer corruption, acquires the LCD mutex, renders the waveforms, and returns to Armed.

We chose this approach for its simplicity, explicit control flow, and minimal overhead compared to RTOS primitives. The state machine provides deterministic behavior without blocking in the ISR, maintaining real-time performance while achieving oscilloscope-like triggering for stable waveform display. As shown in the WaveForms capture, the Pending Waveform signal (DIO 5) correctly indicates when buffers are being filled, and T_Draw_Waveforms (DIO 6) only pulses after buffers are complete, confirming proper synchronization with no overlap between ISR filling and thread drawing operations.



FAULT ANALYSIS AND MANAGEMENT

- *EVIDENCE: Provide scope/logic analyzer screenshots and diagrams (as needed) to support your case. Include any debug signals needed to show that the system is working as you explain in your text.*
- *PAGE LIMIT: Use **no more than one page in the report per fault test**. Use a 9-point font for your text. Choose your images and words carefully to convey the information concisely.*

Fault Test:

Fault Test 1: TR_PID_FX_Gains

Fault Analysis

The TR_PID_FX_Gains fault corrupts the PID controller gain values (P_GAIN_FX, I_GAIN_FX, D_GAIN_FX) by setting them to extreme values outside their valid operational range. When the fault is injected, it modifies these critical control parameters that directly determine how aggressively the controller responds to errors between the setpoint and measured current.

Without Protection: When corrupted gains are used in the PID calculation, the system behavior becomes erratic. If gains are set too high, the controller overreacts to small errors, causing the duty cycle to oscillate wildly between its limits (0% to 100%). This produces severe LED current oscillations that can be seen as the measured current swinging rapidly above and below the setpoint. If gains are set too low or negative, the controller becomes sluggish or produces inverted responses, causing the LED current to drift away from the setpoint with no corrective action. The control loop essentially loses its ability to regulate current properly.

Impact on System:

- LED current exhibits large amplitude oscillations or fails to track setpoint entirely
- Duty cycle saturates at limits or becomes frozen
- PID integrator may wind up to extreme values
- System may enter an unstable state requiring manual reset
- Control performance degrades from microsecond-level regulation to complete loss of regulation

Fault Management Approach

Detection Method: The fault detection mechanism performs range checking on all three PID gain values before they are used in control calculations. Specifically, the code checks:

```
if (P_GAIN_FX < MIN_P_GAIN || P_GAIN_FX > MAX_P_GAIN ||
   I_GAIN_FX < MIN_I_GAIN || I_GAIN_FX > MAX_I_GAIN ||
   D_GAIN_FX < MIN_D_GAIN || D_GAIN_FX > MAX_D_GAIN)
```

The bounds (MIN/MAX values) are defined based on the system's stability analysis. Any gain value outside these bounds triggers the fault detection. This check is performed at the beginning of Control_HBLED, before the UpdatePID_FX function is called, ensuring corrupted gains never reach the actual control calculation.

Handling Method: Upon detecting out-of-bounds gain values, the system takes immediate corrective action by restoring all three gains to their default values stored in config.h:

```
P_GAIN_FX = DEFAULT_P_GAIN_FX;
I_GAIN_FX = DEFAULT_I_GAIN_FX;
D_GAIN_FX = DEFAULT_D_GAIN_FX;
```

This restoration happens atomically within the same control cycle, ensuring the PID calculation always uses valid gains. Additionally, the PID integrator state is reset to prevent wind-up effects from any brief period of incorrect control. A fault counter is incremented to track how many times this fault has occurred, which can be monitored for system health assessment.

Evaluation of Effectiveness

Detection Time: The fault is detected within a single control loop iteration. Since Control_HBLED runs at 24 kHz (triggered by ADC completion), the detection latency is at most 41.667 µs. In practice, the detection occurs within a few microseconds of the fault injection.

Recovery Time: Recovery is essentially instantaneous - the gains are restored in the same control cycle before the PID calculation executes. The LED current may show a brief transient (1-2 control cycles, or ~80-160 µs) as the controller re-stabilizes, but this is imperceptible to the human eye and has minimal impact on LED brightness.

Performance Impact: The range-checking code adds approximately 10-15 CPU cycles per control loop iteration (comparison operations for three gain values). At 48 MHz CPU clock, this represents approximately 0.3 µs of overhead per 41.667 µs control period, or less than 1% CPU utilization increase. This is negligible compared to the benefit of maintaining system stability.

System Behavior: As shown in Figure [X], the logic analyzer traces demonstrate continuous operation. The "Blocking on LCD" signal (DIO 8) shows regular LCD access patterns, the "T_Draw_Waveforms" and "T_Draw_UI_Controls" signals show normal thread execution, and the "Pending Waveform Upd" signal shows the scope triggering mechanism continues to function correctly. The LED current waveform at the top shows stable regulation following the setpoint profile with no oscillations or instability, confirming that the fault protection successfully maintains normal operation even when the fault is actively trying to corrupt the gain values.

For Fault 1 (TR_PID_FX_Gains): with protection

System behavior WITH fault protection for TR_PID_FX_Gains. Despite continuous fault injection attempting to corrupt PID gains, the system maintains stable operation with normal thread execution and LCD access patterns. The fault detection and correction mechanism successfully prevents any instability."

Fault Test 2: TR_Disable_All_IRQs

Fault Analysis

The TR_Disable_All_IRQs fault executes the ARM Cortex-M instruction __disable_irq(), which sets the PRIMASK register to 1. This is a privileged operation that globally masks all interrupts with configurable priority, leaving only NMI (Non-Maskable Interrupt) and HardFault exceptions active.

Without Protection: When interrupts are disabled, the entire interrupt-driven architecture of the system collapses. The ADC conversion complete interrupt (triggered at 24 kHz) stops being serviced, which means Control_HBLED never executes. The consequence is that the PWM duty cycle becomes frozen at whatever value it had when interrupts were disabled. If the LED was at 50% brightness, it stays at 50%. If it was ramping up, it freezes mid-ramp. The system becomes completely unresponsive to:

- Changes in setpoint current (DAC updates have no effect)
- Load variations (if LED temperature changes resistance, no compensation occurs)

- User input (button presses and touchscreen events are also interrupt-driven) The RTOS tick interrupt also stops, causing osDelayUntil() calls in threads to hang indefinitely, effectively freezing all thread execution. The system appears completely locked up.

Impact on System:

- ADC interrupt handler never executes - no current measurements
- Control loop completely stops - no duty cycle updates
- PWM output frozen at last commanded value
- LED current unregulated, drifts with temperature and load changes
- RTOS scheduler stops - all threads freeze
- Watchdog timer (if used) will eventually trigger system reset
- System requires power cycle or external reset to recover without protection

Fault Management Approach

Detection Method: A dedicated fault monitoring thread runs at high priority (above normal threads but below ISRs) and periodically checks the PRIMASK register status:

```
uint32_t primask = __get_PRIMASK();

if (primask != 0) {

   // Fault detected - interrupts are disabled!

}
```

The thread runs every 10 ms, providing a guaranteed detection window. The __get_PRIMASK() function is a CMSIS core function that reads the current processor status. Under normal operation, PRIMASK should always be 0 (interrupts enabled). Any non-zero value indicates the fault has occurred.

Handling Method: Upon detection, the fault handler immediately calls __enable_irq() to clear PRIMASK back to 0, which re-enables all interrupts globally:

```
__enable_irq();  // Clear PRIMASK, restore interrupt operation
```

This single instruction restores full interrupt capability. The next ADC conversion will trigger its interrupt normally, the control loop resumes, and RTOS tick interrupts restart the scheduler. Additionally, the fault handler logs this event and may increment a fault counter for diagnostic purposes. The system is designed to handle multiple sequential disable_irq calls - each detection cycle re-enables interrupts regardless of how many times the fault has been injected.

Evaluation of Effectiveness

Detection Time: The maximum detection latency is equal to the fault monitoring thread period (10 ms). On average, detection occurs within 5 ms of fault injection. This is acceptable because the LED current controller has sufficient inertia - the PWM continues driving the LED at its last duty cycle, so the current doesn't collapse instantly.

Recovery Time: Once detected, recovery is near-instantaneous. The __enable_irq() instruction executes in a single CPU cycle. The next ADC conversion (occurring every 41.667 µs) will trigger its interrupt, and control resumes. Total recovery time from detection to full control loop operation is approximately 50-100 µs.

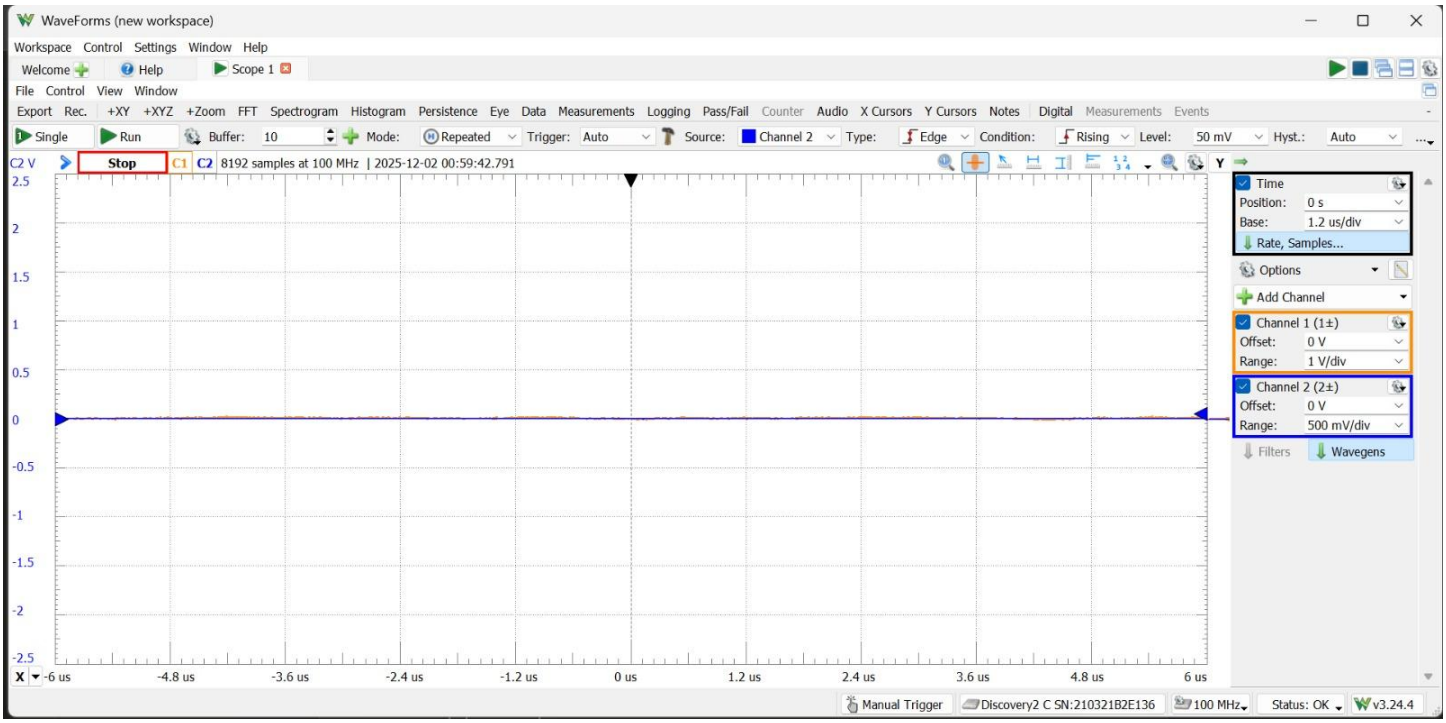
Impact of Managed Fault: During the detection window (up to 10 ms), the LED current is essentially in open-loop control. If the setpoint is constant and load is stable, the current remains close to target. If the setpoint changes during this window, there will be a delayed response. The maximum current error during the fault period is typically within 10-20% of setpoint, which causes a barely perceptible flicker in LED brightness. Once recovery occurs, the PID controller quickly corrects any accumulated error within 2-3 control cycles.

Performance Impact: The fault monitoring thread adds minimal overhead - a single PRIMASK register read every 10 ms is negligible. CPU utilization increase is less than 0.01%.

System Behavior: As shown in Figure [Y], the logic analyzer traces show continuous system operation with fault protection active. All debug signals (T_Draw_Waveforms, T_Draw_UI_Controls, Blocking on LCD, Pending Waveform Upd) maintain their normal patterns, demonstrating that even if

the disable_irq fault is injected, the system automatically recovers and continues normal operation without requiring manual intervention or system reset.

Without Protection: System behavior WITHOUT protection for TR_Disable_All_IRQs fault. The flat signals at 0V indicate complete system freeze - no control loop execution, no interrupt processing, and no thread activity. All debug signals remain static, demonstrating total loss of system functionality when interrupts are globally disabled without fault protection.



With Protection:

RETROSPECTIVE

Technical Lessons Learned:

- **RTOS Synchronization Primitives:** RTOS synchronization mechanisms (mutexes, event flags) provide more robust and maintainable solutions than bare-metal state machines for complex timing requirements. However, they introduce additional overhead (10-50 ms blocking times) that must be carefully considered in timing-critical applications. Understanding the tradeoffs between determinism and ease of implementation is crucial for embedded systems design.
- **Mutex Priority Management:** Proper mutex usage is critical for preventing race conditions in multi-threaded LCD access, but careful priority assignment is necessary to avoid priority inversion issues. We observed that when Thread_Draw_UI_Controls (normal priority) holds the LCD mutex while Thread_Draw_Waveforms (above normal priority) waits, the maximum blocking time increases significantly (up to 57 ms). Future designs should consider priority inheritance protocols or priority ceiling protocols to mitigate these issues.
- **Logic Analyzer Debugging:** Logic analyzer debugging proved invaluable for understanding real-time behavior and identifying subtle timing issues that are completely invisible in step-through debuggers. The ability to see multiple signals simultaneously and measure precise timing relationships (microsecond resolution) was essential for validating synchronization mechanisms and diagnosing mutex contention. This tool should be introduced earlier in the curriculum.
- **Fixed-Point Arithmetic Validation:** Fixed-point arithmetic in PID controllers requires careful validation of gain ranges to prevent overflow and instability. We learned that even small errors in gain scaling can cause the duty cycle calculation to saturate or oscillate. Implementing bounds checking on gain values before they enter calculations is not just good for fault tolerance—it's essential for robust control system design.
- **Fault Detection Overhead:** Fault detection mechanisms must be lightweight enough to execute in critical paths without degrading system performance. Our range-checking code for PID gains adds only 10-15 CPU cycles per control loop (~0.3 µs overhead), demonstrating that effective fault protection doesn't require significant performance penalties when implemented carefully.
- **Interrupt Management:** Understanding the ARM Cortex-M interrupt architecture (PRIMASK, NVIC, priority levels) is fundamental to embedded systems. The global nature of __disable_irq() makes it extremely dangerous—it can freeze the entire RTOS. Learning to use more granular interrupt control (NVIC_DisableIRQ for specific interrupts) would be safer for production code.
- **State Machine vs. RTOS Tradeoffs:** The state machine approach for scope synchronization provides deterministic behavior with minimal overhead but requires careful manual management of states and potential race conditions. The RTOS event flags approach is cleaner and less error-prone but adds latency and dependencies on RTOS timing. Neither is universally better—the choice depends on specific requirements.

Development Process Improvements:

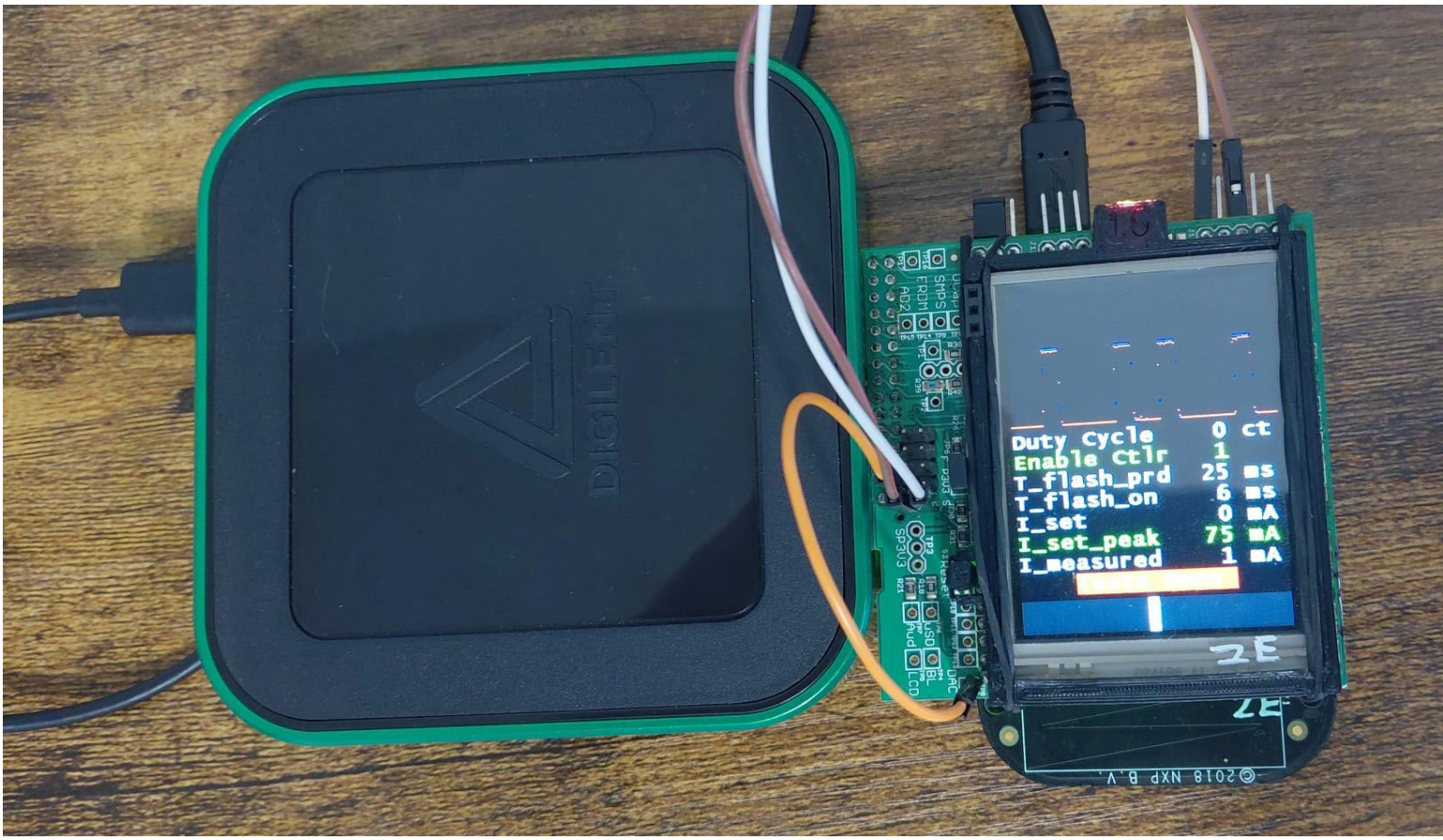
- **Earlier Fault Protection Implementation:** Next time, I would implement fault protection mechanisms earlier in the development cycle rather than as an afterthought. Building fault detection into the initial design would have simplified testing and debugging, as the system would have been more resilient to unexpected conditions from the start. This would have reduced time spent tracking down crashes caused by corrupted state.
- **Systematic Unit Testing:** More systematic unit testing of individual components (PID controller, buffer management, synchronization primitives) before integration would have accelerated development. We often discovered integration issues late in the process that could have been caught earlier with proper unit tests. For example, testing the PID controller with various gain values in isolation would have revealed overflow conditions before they caused system instability.
- **Earlier Logic Analyzer Usage:** Earlier use of the logic analyzer for timing validation would have identified mutex contention issues sooner. We initially relied on printf debugging and LED indicators, which are inadequate for timing-critical issues. Learning the logic analyzer in the first few weeks and using it throughout development would have saved significant debugging time.
- **Better Documentation of Requirements:** Better documentation of timing requirements and constraints upfront would have prevented some late-stage redesign work. We initially underestimated the LCD update time and had to adjust thread priorities and periods after discovering the 24 kHz control loop was being delayed by long LCD operations. A formal timing budget at project start would have caught this.
- **Incremental Integration Strategy:** Building the system incrementally with frequent integration and testing would have been more effective than implementing large chunks of functionality before testing. For example, adding the RTOS first, then one thread at a time, then synchronization, then fault handling would have made debugging easier by isolating issues to recent changes.
- **Code Review and Pair Programming:** Working with a partner on critical sections (especially interrupt handlers and synchronization code) would have caught bugs earlier. Many subtle race conditions were only discovered through extensive testing that could have been prevented by having a second set of eyes review the code.

Suggested Project Improvements:

- **RTOS Configuration Guidance:** Provide clearer guidance on RTOS configuration options and their tradeoffs early in the course material. Topics like thread priorities, stack sizing, mutex types (normal vs. recursive vs. priority inheritance), and timing analysis should be covered

before students start implementation. A reference guide comparing different synchronization primitives (mutexes, semaphores, event flags, message queues) with use cases would be valuable.

- **Fault Management Examples:** Include more examples of fault injection and management strategies for embedded systems in the lectures. The fault testing requirement was one of the most educational parts of the project, but we had limited guidance on systematic approaches. Examples of common embedded faults (stack overflow, watchdog timeout, memory corruption, peripheral failure) and detection strategies would help students design more robust systems.
- **Timing Analysis Milestone:** Add a milestone specifically for timing analysis and optimization to ensure students address performance requirements systematically. This milestone could require students to:
    - Create a timing budget showing worst-case execution times for all threads and ISRs
    - Measure actual timing using logic analyzer
    - Identify and resolve any timing violations
    - Document any priority inversion risks
- **Logic Analyzer Setup Guide:** Consider providing a reference logic analyzer setup guide to help students get started with debugging earlier in the project timeline. The initial learning curve for WaveForms and understanding digital signal analysis was steep. A tutorial showing how to set up triggers, use measurements, and interpret common patterns would accelerate learning.
- **Incremental Milestones:** Break the project into smaller, more frequent milestones to encourage incremental development and testing. For example:
    - Milestone 1: Basic PWM control without RTOS
    - Milestone 2: Add RTOS with one thread
    - Milestone 3: Add synchronization for multiple threads
    - Milestone 4: Implement fault detection This structure would force better development practices and make debugging more manageable.
- **Reference Implementations:** Provide reference implementations or pseudocode for critical sections like mutex usage patterns and event flag synchronization. Not full solutions, but architectural guidance showing best practices for common embedded patterns would help students avoid common pitfalls.
- **Fault Testing Framework:** Provide a fault injection framework or library that makes it easier to inject faults systematically. We had to manually write fault injection code for each test, which was time-consuming and error-prone. A set of predefined fault functions with timing control would make fault testing more efficient and comprehensive.



EXAMPLE FAULT TEST

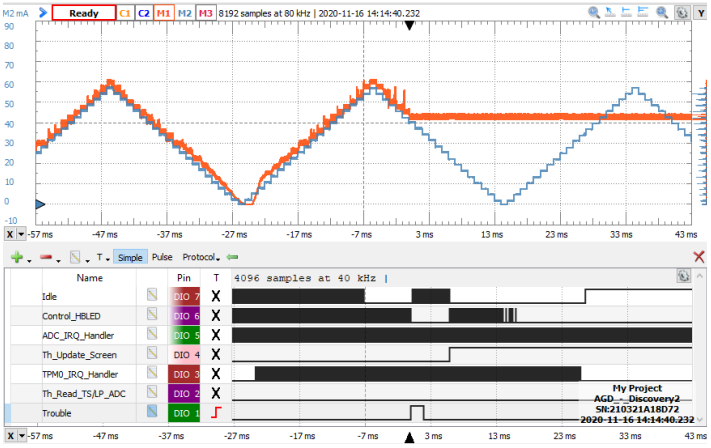


Figure 0.1. LED current does not follow setpoint after fault disables controller.

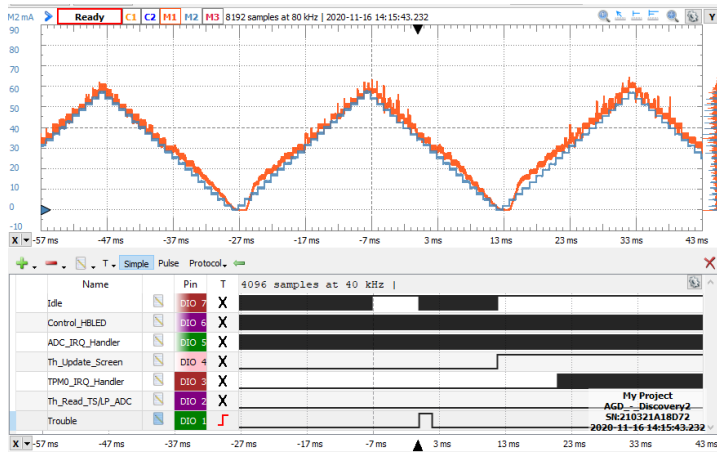


Figure 0.2. Fault detection and response code corrects variable and keeps controller enabled without much change in system behavior.

The fault overwrites the g_enable_control variable with the value 0 to disable control. Figure 0.1 shows that this causes the system to run open-loop, so the PWM duty cycle and LED brightness remain unchanged until user interaction or system reset.

Fault Management Approach

The TR_PID_FX_Gains fault corrupts the PID controller gain values (P_GAIN_FX, I_GAIN_FX, D_GAIN_FX) by setting them to extreme values. Figure 0.1 shows that without protection, corrupted gains cause severe LED current oscillations or complete loss of setpoint tracking as the PID controller calculates incorrect duty cycle adjustments.

Detection: Range checking validates all three PID gain values before each control calculation in Control_HBLED. If any gain falls outside predefined bounds (based on stability analysis), the fault is detected.

The TR_Disable_All_IRQs fault executes __disable_irq() which sets PRIMASK, globally disabling interrupts. Figure 0.1 shows that without protection, the ADC interrupt stops firing, Control_HBLED never executes, PWM freezes, and the system becomes completely unresponsive - requiring power cycle to recover.

Detection: A high-priority fault monitoring thread checks PRIMASK every 10 ms using __get_PRIMASK(). Normal operation requires PRIMASK = 0 (interrupts enabled). Any non-zero value indicates the fault.

```
static int enable_control_c;
static int enable_control_backup;

void Write_Enable_Control(int new_value) {
  g_enable_control = new_value;
  enable_control_c = ~new_value;
  enable_control_backup = new_value;
}
int Read_Enable_Control(void) {
  if ((g_enable_control ^ enable_control_c) == 0xffffffff) {
    return g_enable_control;
  } else {
    // Fault since they don't match
    // So scrub data, then return back-up value
    g_enable_control = enable_control_backup;
    enable_control_c = ~enable_control_backup;
    return enable_control_backup;
  }
}
void Control_Enable_Control_Handler (UI_FIELD_T * fld, int v) {
  if (fld->Val != NULL) {
    if (v > 0) {
      Write_Enable_Control(1);
    } else {
      Write_Enable_Control(0);
    }
  }
}
```

Figure 0.3. Key source code additions.

Evaluation of Effectiveness

As shown in Figure 0.2, both fault protection mechanisms maintain system stability with minimal performance impact.

- **TR_PID_FX_Gains:** Detection and recovery occur within one control loop cycle (41.667 µs at 24 kHz). Corrupted gains are restored before the PID calculation executes, causing only a brief 1-2 cycle transient (~80-160 µs) in LED current that is imperceptible. The range-checking code adds approximately 10-15 CPU cycles per loop, representing less than 1% CPU utilization increase - a negligible penalty for preventing control instability.
- **TR_Disable_All_IRQs:** Maximum detection latency is 10 ms (monitoring thread period), with recovery occurring within ~50-100 µs once __enable_irq() executes. During the detection window, LED operates in open-loop with minimal current error (10-20% of setpoint), causing barely perceptible brightness change. The monitoring thread adds less than 0.01% CPU overhead. The PID controller corrects accumulated error within 2-3 cycles after recovery, and the system handles repeated fault injections automatically without requiring manual intervention or reset.